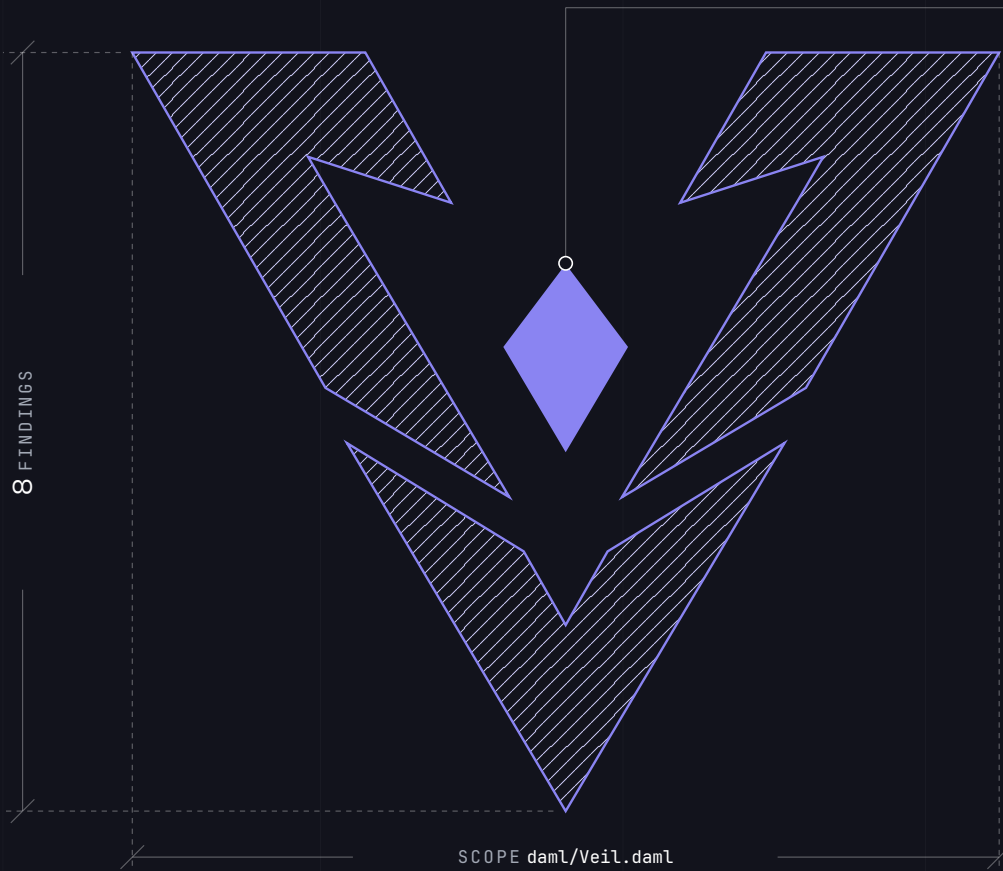


COMMIT 2455470



SECURITY REVIEW

Veil.

September 23rd 2026 - September 28th 2026

Contents.

01	About Valves Security	3
02	Disclaimer	3
03	Risk Classification	3
04	Protocol Summary	4
05	Executive Summary	4
06	Findings	5
	■ High Findings	5
	[H-01] Lender supplies the liquidation price, so it can seize the collateral of any healthy loan	5
	[H-02] Regulator is a free choice of the lender	5
	■ Medium Findings	6
	[M-01] Holdings have no issuer, so anyone can mint fake cash and collateral	6
	[M-02] Liquidation seizes all collateral, including value above the debt	6
	■ Low Findings	7
	[L-01] Loan terms are not bounded, allowing loans that can be liquidated at once	7
	[L-02] Lender can archive the shared <code>LoanClosed</code> record alone	8
	[L-03] Borrowers cannot reject offers, and offers never expire	8
	[L-04] Repay requires the exact amount, so the change logic never runs	9

SECTION 01

About Valves Security.

Valves Security is an independent smart contract security firm founded by Veselin and Valeri. We work with security researchers specializing in Solidity, Rust and DAML. Every engagement is led by both co-founders with senior security researchers: we review the code line by line, substantiate every reported issue and remain involved through remediation.

To request a security review, visit valvessecurity.com or contact @ValvesSec on X.

SECTION 02

Disclaimer.

This report reflects a time-boxed review of the code at the commit listed in the Executive Summary, limited to the security of the files in scope. A security review cannot guarantee the absence of vulnerabilities, and this report is neither an endorsement of the project, its team or any associated token, nor investment advice. Changes made after the reviewed commit may introduce new issues; we recommend follow-up reviews and an ongoing bug bounty program.

SECTION 03

Risk Classification.

LIKELIHOOD / IMPACT	IMPACT: HIGH	IMPACT: MEDIUM	IMPACT: LOW
High	■ Critical	■ High	■ Medium
Medium	■ High	■ Medium	■ Low
Low	■ Medium	■ Low	■ Low

Impact

- **High** — leads to a significant material loss of assets in the protocol or a group of users.
- **Medium** — leads to a moderate material loss of assets in the protocol, moderately harms a group of users or causes some disruption of the protocol's functionality.
- **Low** — no funds at risk.

Likelihood

- **High** — the attack path is possible with reasonable assumptions. The issue is almost certain to happen.
- **Medium** — only a conditionally incentivized attack vector, but still relatively likely.
- **Low** — has unlikely assumptions or requires a significant stake by the attacker.

SECTION 04

Protocol Summary.

Veil is a confidential, repo-style lending protocol on Canton, written in DAML. A lender privately offers short-term loans against tokenized collateral such as T-Bills, money-market fund units or real Canton Coin, and the borrower's collateral is locked atomically when they accept. Canton's privacy model means only the lender, borrower, issuer and an optional regulator can see the position, and the contracts enforce valuer-attested prices, margin calls, collateral top-ups or substitutions, and repayment or liquidation at maturity.

SECTION 05

Executive Summary.

OVERVIEW

PROJECT Veil	REVIEW PERIOD September 23rd 2026 - September 28th 2026
REPOSITORY github.com/no-witness-labs/veil-lite-hackathon	COMMIT 2455470f2521b1ce295252178f11507f8c881321

SCOPE

ID	FILES IN SCOPE	NSLOC
01	daml/Veil.daml	943

SECTION 06

Findings.

■ High Findings

[H-01] Lender supplies the liquidation price, so it can seize the collateral of any healthy loan

HIGH

FIXED

Description

Only the lender can call `Liquidate`, and the only check uses `currentCollateralValue`, a number the lender passes in (`daml/Veil.daml`, lines 181 to 189):

```
1     currentCollateralValue : Decimal
2     controller lender
3     ...
4     (principal / currentCollateralValue) * 100.0 >= liquidationThresholdLtv
```

The lender can pass any low value, and the check passes. No one else confirms the price, and it is not recorded.

Impact

Right after the loan starts, the lender can liquidate at a value of `1.0` and take all 150 TBILL. The borrower keeps the 100 principal but loses collateral worth 150, which is 45 worse than repaying. The borrower cannot stop this.

Recommendation

Read the collateral value from a contract signed by someone other than the lender (e.g. a valuation agent both parties agree on), and `fetch` it inside `Liquidate`. Reject stale values using `getTime`, and record the value on `LoanClosed`.

[H-02] Regulator is a free choice of the lender

HIGH

FIXED

Description

`regulator` is a plain argument to `MakeOffer` (line 19). It is copied onto the offer, the loan and the closed record (lines 85, 141 and 222), and nothing checks it. The lender can name itself, the borrower, or any other party.

Impact

With `regulator = lender`, the intended regulator never sees the loan. With any other party named, that party sees the borrower's identity and terms. The borrower sees the regulator on the offer before accepting, and the docs call the regulator "optional" ([README.md](#), line 15), so this affects oversight only when the borrower accepts a bad offer.

Recommendation

If regulator oversight is meant to be required, take the regulator from a regulator-signed contract and assert `regulator != lender && regulator != borrower`.

■ Medium Findings

[M-01] Holdings have no issuer, so anyone can mint fake cash and collateral

MEDIUM

FIXED

Description

`CashHolding` and `CollateralHolding` are signed only by their owner ([daml/Veil.daml](#), lines 10 and 64), so anyone can create one with any amount. `Accept` and `Repay` only check the fields the creator chose, so they cannot tell real holdings from fake ones.

`LoanOffer` is signed only by the lender (line 84), so a lender can also create an offer directly without funding it.

Impact

- **Fake collateral:** a borrower creates 150 TBILL and borrows the lender's real 100.
- **Fake repayment:** a borrower creates 105 cash, repays, and gets the real collateral back.
- **Unfunded offer:** a lender creates an offer directly, then calls `Withdraw` to receive 100 it never paid in.

This would be Critical with real assets. It is Medium because the docs say this demo uses no real tokenized assets.

Recommendation

Make holdings signed by an issuer (`signatory issuer, owner`), or use the Canton Token Standard `Holding`. Lock the principal and collateral in escrow contracts and transfer them, instead of archiving holdings and creating new ones later.

[M-02] Liquidation seizes all collateral, including value above the debt

MEDIUM

FIXED

Description

`Liquidate` always gives the lender the full `collateralQuantity` (line 189), even when the collateral is worth more than the debt (`principal + interest`). Nothing is returned to the borrower.

Under GMRA-style close-out netting, value above what is owed goes back to the borrower. Here liquidation is triggered by an LTV breach, not a default, and the borrower has no chance to add collateral first.

Impact

At the standard 90% threshold, an honest liquidation can happen only at values up to 111.11, against a debt of 105. So the borrower loses at most about 6.

Recommendation

Seize only `ceiling(collateralQuantity × (principal + interest) / currentCollateralValue)` units and return the rest to the borrower. Otherwise, document full seizure as a deliberate term.

■ Low Findings

[L-01] Loan terms are not bounded, allowing loans that can be liquidated at once

LOW FIXED

Description

The only checks on loan terms are these (`daml/Veil.daml`, lines 87 to 88 and 143 to 144):

```
1    ensure principal > 0.0 && interest >= 0.0 && collateralQuantity > 0.0
2    && liquidationThresholdLtv > 0.0
```

The threshold, which is a percentage, has no upper limit, and nothing checks that the loan starts below it. `principal` and `interest` have no upper limits either.

Impact

- **Low threshold:** with a threshold of 60, or 0.9 entered as a fraction instead of a percentage, the loan can be liquidated as soon as it starts, even at a fair value of 150. The borrower loses 150 of collateral against a 105 debt.
- **Huge values:** with `interest` near the `Decimal` limit (about 1e28), `principal + interest` (line 154) overflows, so `Repay` always fails and the collateral stays locked.

The borrower must accept these terms, which makes both cases less likely.

Recommendation

Add `liquidationThresholdLtv <= 100.0` and sensible upper limits on `principal` and `interest`. Once a trusted price exists, check in `MakeOffer` or `Accept` that the starting LTV is below the threshold.

[L-02] Lender can archive the shared `LoanClosed` record alone

LOW FIXED

Description

`LoanClosed` is signed by both lender and borrower as a shared settlement record, but only the lender controls `Dismiss` (`daml/Veil.daml`, lines 221 to 229):

```
1    signatory lender, borrower
2    observer regulator
3    ...
4    choice Dismiss : ()
5        controller lender
6    do
7        pure ()
```

The borrower has no way to archive it. The record also does not store the liquidation price, the threshold or the closing time (lines 208 to 219), so a liquidation cannot be checked afterwards.

`Dismiss` exists for the demo reset flow (`docs/ARCHITECTURE.md`, lines 141 to 143).

Impact

The lender can remove the borrower's proof of repayment, or the record of an abusive H-01 liquidation, from the borrower's and regulator's active contracts. The archive event stays in each party's transaction history until pruning, and no funds move.

Recommendation

Use `controller lender, borrower`, or remove `Dismiss` outside the demo build. Add `liquidation Mark : Optional Decimal` and `closedAt : Time` to `LoanClosed`.

[L-03] Borrowers cannot reject offers, and offers never expire

LOW FIXED

Description

`MakeOffer` accepts any `borrower` and `regulator` (lines 16 to 43). Only the lender can remove an offer (`Withdraw`, line 118). The borrower and regulator cannot, and offers have no expiry (L-01).

Impact

Unwanted offers pile up in the borrower's and regulator's views.

Recommendation

Add a borrower `Reject` choice and an offer expiry, and fix M-01.

[L-04] Repay requires the exact amount, so the change logic never runs

LOW FIXED

Description

Line 157 requires `cash.amount == repayment`, so the branch that returns change (lines 160 to 164) can never run. The code comments, [README.md](#) and [docs/ARCHITECTURE.md](#) all say overpayment is accepted and change is returned. There is also no way to merge cash holdings, and the frontend computes the repayment with JavaScript `Number` addition ([frontend/src/App.tsx](#), line 236).

Impact

A borrower holding 110 cash has to split it before repaying. Today this is extra friction and a mismatch with the docs. With real holdings (after M-01), cash spread across several holdings could block repayment. Changing line 157 to `>=` would reopen the privacy leak fixed in commit [9362877](#) (“close cash-balance divulgence on repay”).

Recommendation

Either remove the change branch and document that repayment must be exact, or add a [Merge](#) choice and accept `>=`, sending the change in a way the regulator does not see. Compute the repayment amount as a decimal string in the frontend.